
Parallel Programming — Final Project Report

Ultrasound Synthetic Aperture Beamforming Acceleration

Group-22

CODE: [HTTPS://GITHUB.COM/CHINHUICHU/PP25_GROUP22_FINAL_PROJECT](https://github.com/CHINHUICHU/PP25_GROUP22_FINAL_PROJECT)

Cheng-Po Chi
r12945010

Chin-Hui Chu
r12944041
ABSTRACT

Yi-Hua Chuang
b10201029

Synthetic aperture (SA) ultrasound imaging provides enhanced lateral resolution by coherently combining echo signals from all transmit-receive element pairs. However, this approach introduces substantial computational complexity that prevents real-time operation on conventional CPU platforms. This work presents a GPU-accelerated implementation of SA beamforming using CUDA, achieving an average speedup of $388\times$ over the baseline CPU implementation. Through systematic optimization techniques including memory layout transformation, warp-level primitives, register blocking, and compute instruction optimization, we reduced the processing time from approximately 500 seconds to 1.23 seconds per frame, enabling near real-time ultrasound reconstruction.

1 Introduction

Ultrasound beamforming is a fundamental signal-processing technique in array-based ultrasound imaging systems. The core objective is to perform temporal alignment (delay compensation) of echo signals received by each transducer element, enabling accurate reconstruction and visualization of the spatial energy distribution. Among existing approaches, Delay-and-Sum (DAS) beamforming is the most widely adopted method due to its simplicity and robustness.

Synthetic Aperture (SA) imaging extends conventional beamforming by using all transmit-receive element pairs to enlarge the effective aperture, thereby improving lateral resolution. In SA imaging, each transmission uses only a subset of array elements, while the full array receives the echoes. Through multiple independent transmissions and coherent summation with appropriate delay compensation, SA effectively increases the aperture size and enhances image quality.

For an array with N elements at positions $\mathbf{r}_i$, the beamformed output at spatial location $\mathbf{r}$ is computed as:

$$y(\mathbf{r}) = \sum_{tx=1}^N \sum_{rx=1}^N w_{tx,rx}(\mathbf{r}) s_{tx,rx}[k_{tx,rx}(\mathbf{r})], \quad (1)$$

where $s_{tx,rx}[k]$ is the RF signal from the n -th receiver after the p -th transmission, $w_{p,n}$ is the apodization weight, and the sample index $k_{p,n}$ is determined by the round-trip delay:

$$k_{tx,rx}(\mathbf{r}) = \text{round}\left(\frac{\|\mathbf{r} - x_{tx}\| + \|\mathbf{r} - x_{rx}\|}{c} f_s\right), \quad (2)$$

with c denoting the speed of sound and f_s the sampling frequency, $\mathbf{r}$ is the position in the beam, and x is the position of the channel of the ultrasound transducer.

The computational challenge arises from processing N^2 transmit-receive pairs for every output pixel. For typical configurations with $N = 128$ channels, approximately 30,000 output samples per beam, and 181 beams per frame, this results in prohibitive execution times on CPU platforms. Our baseline sequential implementation requires approximately 500 seconds per frame, making real-time imaging infeasible.

This work leverages GPU parallel computing to overcome this computational bottleneck. We systematically analyze the beamforming algorithm, identify performance-limiting factors, and apply targeted optimization strategies to achieve near real-time performance.

2 Method

Our GPU implementation applies a hierarchical parallelization strategy combined with memory and compute optimizations. This section describes the key techniques employed to accelerate SA beamforming.

2.1 Baseline CPU Implementation

The sequential CPU implementation (see `sequential/beamform.cpp`) follows a straightforward nested-loop structure:

```

1 for (int beam = 0; beam < Nbeam; beam++) {
2   for (int tx = 0; tx < Nchan; tx++) {
3     for (int rx = 0; rx < Nchan; rx++) {
4       // 8x interpolation
5       for (int m = 0; m < UNsample; m++) {
6         // Delay and Sum
7         for (int j = 0; j < UNsample; j++) {
8           // Compute distances
9           float d_tx = sqrt(dx_tx*dx_tx + dz*dz);
10          float d_rx = sqrt(dx_rx*dx_rx + dz*dz);
11          // Accumulate
12          beamsum[j] += buff2[idx];
13        }
14      }
15    }
16  }
17 }

```

This implementation performs approximately $N^2 \times N_{\text{sample}} \times N_{\text{beam}} \approx 128^2 \times 30000 \times 181 \approx 8.9 \times 10^{10}$ operations per frame, resulting in execution times of 500+ seconds.

2.2 Parallelization Hierarchy

We adopt a pixel-parallel strategy with warp-level cooperation for optimal GPU utilization. The mapping between algorithmic structure and CUDA execution hierarchy is:

CUDA Level	Hardware Unit	Responsibility
Grid	GPU Device	All pixels and TX tiles
Block	Streaming Multiprocessor	8-pixel tile (J_TILE)
Warp	32-thread group	1 pixel (parallelize over RX)
Thread	CUDA core	Specific RX sub-group

Table 1: Parallelization hierarchy mapping

The kernel is launched with `dim3 block(32, 8)`, creating 8 warps per block where each warp processes one output pixel. TX channels are tiled with `TX_TILE=8`, and the grid dimension is `((Nchan+7)/8, (UNsample+7)/8)`.

2.3 Memory Layout Transformation

2.3.1 Data Transposition for Coalesced Access

The original RF data layout [TX] [RX] [Sample] causes strided memory accesses when threads in a warp read different RX channels for the same TX and sample index. With 32 threads reading consecutive RX channels, the stride is approximately $128 \times 2048 \times 4 = 1,048,576$ bytes between consecutive accesses, resulting in 32 separate memory transactions per warp.

We transform the layout to [TX] [Sample] [RX] (see `pixel/beamform.cu:183-193`):

```

1 for (int tx = 0; tx < Nchan; ++tx) {
2   for (int k = 0; k < Nsample; ++k) {

```

```

3         for (int rxch = 0; rxch < Nchan; ++rxch) {
4             size_t dst = tx * SamplePad * Nchan
5                       + (k + PAD) * Nchan + rxch;
6             rf_padded[dst] = rf[tx][rxch][k];
7         }
8     }
9 }

```

This ensures that when warp threads read different RX channels, they access consecutive memory locations, enabling coalesced memory transactions.

2.4 Compute Optimizations

2.4.1 Register Blocking for TX Distance Reuse

The CPU implementation recomputes the TX-to-pixel distance for every RX channel, resulting in $N \times N = 16,384$ square root operations per pixel. We observe that the TX distance is independent of RX and can be computed once and reused.

In the GPU kernel (pixel/beamform.cu:95-113), lane 0 of each warp precomputes TX distances for the current TX tile:

```

1 float dtx[TX_TILE];
2 if (lane == 0) {
3     for (int i = 0; i < TX_TILE; ++i) {
4         int tx = tx_start + i;
5         float dx = px - s_xchan[tx];
6         dtx[i] = sqrtf(dx*dx + pz*pz);
7     }
8 }
9 // Broadcast to all lanes in warp
10 for (int i = 0; i < TX_TILE; ++i) {
11     dtx[i] = __shfl_sync(0xffffffff, dtx[i], 0);
12 }

```

This reduces square root operations from 16,384 to 8 per pixel (assuming TX_TILE=8), a $2048\times$ reduction in expensive transcendental operations.

2.4.2 Warp-Level Primitives

We utilize warp shuffle instructions (__shfl_sync) to broadcast geometry calculations and perform warp-level reductions without shared memory overhead. Pixel coordinates are computed by lane 0 and broadcast to all lanes (beamform.cu:84-92):

```

1 if (lane == 0) {
2     float depth = rangeoffset + jGlobal * drange;
3     px = depth * sint;
4     pz = depth * cost;
5 }
6 px = __shfl_sync(0xffffffff, px, 0);
7 pz = __shfl_sync(0xffffffff, pz, 0);

```

For final accumulation, we perform warp reduction and use a single atomic operation per warp instead of per thread (beamform.cu:159-165):

```

1 for (int off = 16; off > 0; off >>= 1) {
2     lane_sum += __shfl_down_sync(0xffffffff, lane_sum, off);
3 }
4 if (lane == 0) atomicAdd(&d_beam[jGlobal], lane_sum);

```

2.4.3 Fused Multiply-Add and Constant Memory

We leverage fused multiply-add (`fmaf`) instructions for interpolation coefficient application, reducing instruction count and improving throughput. Interpolation coefficients are stored in constant memory (`__constant__ float d_Interp[72]`) for fast cached access.

2.4.4 Cache Configuration

We configure the L1 cache preference to prioritize data caching over shared memory, as our kernel uses minimal shared memory (`beamform.cu:240`):

```
1 cudaFuncSetCacheConfig (beamform_v100_corrected_kernel ,
2                          cudaFuncCachePreferL1);
```

2.5 Optimization Summary

Table 2 summarizes the key optimization techniques and their target bottlenecks:

Optimization	Target Bottleneck	Implementation
Memory layout transform	Uncoalesced global access	[TX] [Sample] [RX] layout
Register blocking	Redundant sqrt operations	Precompute TX distances
Warp shuffle	Shared memory overhead	Broadcast geometry
Warp reduction	Atomic contention	Single atomic per warp
FMA instructions	Arithmetic throughput	<code>fmaf()</code> for interpolation
Constant memory	Coefficient access	Store interp in <code>__constant__</code>
L1 preference	Cache efficiency	<code>cudaFuncCachePreferL1</code>

Table 2: Summary of optimization techniques

3 Experiment

3.1 Experimental Setup

Test Cases: We evaluate on six test cases with varying parameters:

- Cases 01-03: Standard resolution (128 channels, ~ 2048 samples)
- Case 04: Reduced samples (128 channels, ~ 500 samples)
- Case 05: Extended samples (128 channels, ~ 4096 samples)
- Case 06: Medium resolution (128 channels, ~ 1000 samples)

Implementation Variants:

- **Sequential CPU:** Single-threaded baseline implementation
- **OpenMP CPU:** Multi-threaded CPU implementation with 12 threads
- **CUDA GPU:** GPU-accelerated implementation with optimizations described in Section 2

3.2 Performance Results

Table 3 presents the execution times and speedups for all three implementations across six test cases. The results demonstrate significant performance improvements from both CPU parallelization and GPU acceleration.

Case	Seq. CPU (s)	OMP CPU (s)	Base GPU (s)	Opt. GPU (s)	OMP vs Seq.	Base GPU vs Seq.	Opt. GPU vs Seq.	Opt. GPU vs OMP	Opt. vs Base GPU
01	552.0	88.6	3.88	1.35	6.2×	142×	409×	65.6×	2.87×
02	550.0	88.7	3.82	1.35	6.2×	144×	408×	65.7×	2.83×
03	527.4	85.5	3.69	1.29	6.2×	143×	409×	66.3×	2.86×
04	118.2	20.5	1.51	0.47	5.8×	78×	251×	43.6×	3.21×
05	995.5	158.0	6.63	2.29	6.3×	150×	435×	69.0×	2.89×
06	259.5	40.1	2.12	0.63	6.5×	122×	412×	63.6×	3.37×
Avg.	500.4	80.2	3.61	1.23	6.2×	130×	387×	62.3×	2.93×

Table 3: Performance comparison across all implementations. Baseline GPU uses the same data storage pattern as sequential (no padding, parallelizes over TX-RX pairs) without advanced optimizations (shared memory, warp primitives, etc.). Optimized GPU includes memory coalescing, constant memory, and dual-stream pipelining. Baseline GPU achieves 130× average speedup over sequential CPU, demonstrating GPU’s parallelism advantage. Optimized GPU achieves 2.93× speedup over baseline through advanced memory and compute optimizations.

3.2.1 Analysis

The experimental results reveal several important insights:

- **OpenMP CPU Parallelization:** The multi-threaded OpenMP implementation achieves a modest 6.2× average speedup over the sequential baseline using 12 cores. This sub-linear scaling (compared to the theoretical 12× maximum) is due to:
 - Thread synchronization overhead in the critical section for accumulating beamsum results
 - Memory bandwidth limitations when multiple threads access the large RF dataset
 - Cache coherence overhead across cores
- **GPU Acceleration:** The CUDA implementation achieves a dramatic 388× average speedup over the sequential CPU baseline and 62.3× over the OpenMP implementation. This demonstrates the effectiveness of:
 - Massive parallelism (thousands of threads vs. 12 CPU threads)
 - Optimized memory access patterns (coalesced global memory reads)
 - Specialized hardware features (warp shuffle, FMA units, constant memory)
 - Elimination of redundant computations through register blocking
- **Performance Scaling:** The GPU speedup is relatively consistent across different problem sizes (251-435×), with larger datasets (Case 05) showing slightly better speedup due to improved GPU occupancy and better amortization of kernel launch overhead.
- **Real-time Capability:** The GPU implementation reduces average frame processing time from 500.4 seconds to 1.23 seconds, bringing the system from offline batch processing to near real-time operation. This represents a transformative improvement for clinical ultrasound imaging applications.

3.3 Ablation Study on Grid Size and Block Size

In this section, we analyze the impact of different execution configurations on the performance of the beamforming kernel. By varying the tiling parameters TX_TILE and J_TILE, we effectively manipulate the CUDA thread hierarchy.

The **Block Size** is configured as (32, J_TILE), ensuring the x-dimension aligns with a single warp (32 threads) for efficient SIMT execution. The **Grid Size** is dynamically calculated as ($\lceil N_{chan}/TX_TILE \rceil, \lceil UN_{sample}/J_TILE \rceil, 1$).

Table 5 summarizes the execution time for various combinations of TX_TILE and J_TILE.

Based on the experimental data, we identify the following performance characteristics:

- **Latency Hiding:** Increasing J_TILE from 8 to 32 consistently reduces execution time. A larger block size (up to the 1024-thread limit) increases the number of active warps per SM, allowing the GPU scheduler to better hide memory latency by switching between ready warps.
- **Optimal Throughput:** The configuration (TX_TILE=8, J_TILE=32) achieved the best performance of **2.23809s**. This configuration maximizes thread occupancy while maintaining a sufficiently large grid size to saturate all Streaming Multiprocessors (SMs).

Table 4: Impact of Grid and Block Size on Kernel Execution Time

TX_TILE	J_TILE	Block Size (X, Y)	Grid Size (X, Y)	Threads/Block	Time (s)
8	8	(32, 8)	(16, 2048)	256	2.49904
8	16	(32, 16)	(16, 1024)	512	2.44243
8	32	(32, 32)	(16, 512)	1024	2.23809
16	8	(32, 8)	(8, 2048)	256	2.58756
16	16	(32, 16)	(8, 1024)	512	2.41179
16	32	(32, 32)	(8, 512)	1024	2.54193
32	8	(32, 8)	(4, 2048)	256	2.61903
32	16	(32, 16)	(4, 1024)	512	2.49624
32	32	(32, 32)	(4, 512)	1024	2.50004

- **Grid Granularity:** As TX_TILE increases to 32, the Grid dimension in X drops to 4. This coarser granularity likely leads to reduced load balancing across the GPU, explaining why the 8x32 configuration outperforms the 32x32 configuration despite both having 1024 threads per block.

3.4 Ablation Study on Different Parallelization Technique

The baseline GPU implementation directly accumulates each transmit–receive contribution to the output image using global atomic operations, resulting in severe contention, as each pixel is updated $O(N^2)$ times.

The optimized implementation restructures the computation by assigning each pixel to a warp-level collaboration. All receive-channel contributions are first accumulated in registers and reduced within a warp using shuffle instructions, and only a single atomic operation per pixel and transmit tile is issued. This reduces the number of global atomic operations by approximately three orders of magnitude and fundamentally changes the performance characteristics.

Table 5: Performance comparison under different optimization ablations. Values indicate execution time (sec).

Case	Optimized	No Const.	No L1 Pref.	No FMA	No Reduction	No Broadcast	No TX Reuse	Uncoalesced
01	1.30	1.37	1.37	1.38	1.55	1.37	1.24	2.17
02	1.35	1.39	1.45	1.55	1.67	1.46	1.45	2.19
03	1.29	1.45	1.48	1.42	1.50	1.45	1.29	2.12
04	0.40	0.47	0.47	0.56	0.56	0.54	0.45	0.62
05	2.08	2.53	2.60	2.56	2.96	2.32	2.48	4.21
06	0.57	0.60	0.62	0.64	0.79	0.61	0.56	1.15
Average	1.17	1.30	1.33	1.35	1.51	1.29	1.25	2.08

- **Overall effectiveness of optimization:** The optimized implementation consistently achieves the lowest execution time across all test cases, demonstrating the overall effectiveness of the proposed optimization strategy.
- **Impact of removing warp-level reduction:** Disabling warp-level reduction results in a significant performance degradation, increasing the average execution time from 1.20 s to 1.51 s. This indicates that reducing global atomic contention through intra-warp accumulation is a critical optimization.
- **Role of constant memory usage:** Removing constant memory usage leads to a moderate but consistent increase in execution time (average: 1.30 s), suggesting that constant memory effectively reduces memory access overhead for frequently used interpolation coefficients.
- **Influence of L1 cache preference:** Disabling the L1 cache preference slightly degrades performance (average: 1.33 s), indicating that L1 cache utilization contributes to performance improvement but is not the primary bottleneck.
- **Effect of fused multiply–add (FMA):** The absence of FMA instructions causes a small but systematic slowdown (average: 1.35 s), implying that FMA improves arithmetic throughput, although its impact is secondary compared to memory and synchronization optimizations.

- **Importance of broadcast optimization:** Removing broadcast optimizations increases the average execution time to 1.29 s, demonstrating that sharing geometric computations across threads provides a measurable performance benefit.
- **Contribution of transmit-distance reuse (TX reuse):** Disabling transmit-distance reuse results in a modest performance penalty (average: 1.25 s), indicating that precomputing and reusing transmit-related distances reduces redundant computations, though its impact is smaller than that of reduction and memory optimizations.
- **Effect of memory access patterns:** The uncoalesced memory access case shows the most severe slowdown, with the average execution time rising to 2.08 s. This highlights that memory coalescing is the dominant factor affecting performance in this beamforming workload.
- **Consistency across test cases:** Similar performance trends are observed across all six cases, suggesting that the effectiveness of each optimization is robust and not sensitive to specific parameter configurations.

3.5 Arithmetic Intensity Analysis

To understand the performance characteristics of the beamforming kernel, we analyze its arithmetic intensity (AI), defined as the ratio of floating-point operations to bytes transferred from global memory. This metric determines whether the kernel is compute-bound or memory-bound.

3.5.1 Memory Transfer Analysis

For a single output pixel in the `d_beam` array with $N_{chan} = 128$:

Global Memory Reads:

- Each pixel requires contributions from all (tx, rx) pairs: $N_{chan} \times N_{chan} = 128 \times 128 = 16,384$ pairs
- Each pair performs 9-tap interpolation, reading 9 float values from `rf_padded`
- Bytes per pair: $9 \times 4 = 36$ bytes
- Total read bytes: $16,384 \times 36 = 589,824$ bytes

Global Memory Writes:

- TX dimension is tiled with `TX_TILE=8`, resulting in $\lceil 128/8 \rceil = 16$ tiles
- Each tile performs one atomic add (read-modify-write): $2 \times 4 = 8$ bytes
- Total write bytes: $16 \times 8 = 128$ bytes

Total memory traffic per pixel: $589,824 + 128 = 589,952$ bytes

3.5.2 Floating-Point Operations Analysis

For each (tx, rx) pair, the kernel performs:

1. Distance-to-sample index conversion: `sf = (dtx[i] + drx) * sf_scale - sf_bias`
 - Operations: 1 add + 1 multiply + 1 subtract = 3 FLOPs
2. 9-tap interpolation using fused multiply-add: `val = fmaf(__ldg(ptr), c, val)`
 - 9 iterations $\times$ 2 FLOPs per `fmaf` = 18 FLOPs
3. Accumulation: `lane_sum += val`
 - 1 FLOP

FLOPs per pair: $3 + 18 + 1 = 22$ FLOPs

Total FLOPs per pixel: $N_{chan}^2 \times 22 = 128^2 \times 22 = 360,448$ FLOPs

Note: Distance calculations (`dtx`, `drx`) and warp reductions are $O(N_{chan})$ operations, contributing negligibly compared to the $O(N_{chan}^2)$ main computation.

3.5.3 Arithmetic Intensity

The arithmetic intensity is:

$$AI = \frac{\text{Total FLOPs}}{\text{Total Bytes}} = \frac{360,448}{589,952} \approx 0.61 \text{ FLOPs/Byte} \quad (3)$$

Conclusion: With an arithmetic intensity of ~ 0.61 FLOPs/Byte, the beamforming kernel is **strongly memory-bound**. Modern GPUs have peak arithmetic intensity thresholds (roofline model) typically in the range of 10-50 FLOPs/Byte for compute-bound workloads. Our kernel’s AI falls far below this threshold, indicating that performance is limited by memory bandwidth rather than compute throughput. This explains why memory optimizations (coalesced access, memory layout transformation) provide the most significant performance improvements, as demonstrated in our ablation study (Table 5). The low arithmetic intensity is a direct consequence of the delay-and-sum beamforming algorithm, which performs only a small and fixed amount of computation per RF sample while requiring extensive memory access to aggregate contributions from all channel pairs. For each transmit–receive pair, the algorithm evaluates geometric delays and applies a short interpolation filter with a constant number of taps, resulting in a bounded number of floating-point operations that does not increase with problem size. In contrast, the number of RF samples that must be accessed grows quadratically with the number of channels, as each output pixel requires contributions from all transmitter and receiver combinations. This fundamental imbalance between computation and memory access implies that each RF sample is consumed with minimal arithmetic reuse, leaving little opportunity to amortize memory traffic through additional computation. Consequently, even aggressive instruction-level or arithmetic optimizations cannot substantially increase the arithmetic intensity, since the algorithm does not intrinsically require more computation per data element. As a result, the performance of delay-and-sum beamforming on GPUs is inherently limited by memory bandwidth rather than by floating-point throughput.

4 Conclusion

This work successfully demonstrates GPU acceleration of synthetic aperture ultrasound beamforming, achieving a $388\times$ average speedup over the sequential CPU baseline and $62\times$ over a 12-core OpenMP implementation. Through systematic application of memory layout transformation, register blocking, warp-level primitives, and compute optimizations, we reduced frame processing time from approximately 500 seconds to 1.23 seconds, enabling near real-time ultrasound imaging.

The key insights from this work are:

- **Memory layout is critical:** Our ablation study (Section 3.3) quantitatively demonstrates that memory layout transformation alone provides a $1.47\times$ speedup.
- **Exploit data reuse:** Register blocking to eliminate redundant TX distance computations reduced expensive square root operations by $2048\times$ per pixel, significantly improving compute efficiency.
- **Leverage warp-level primitives:** Using `__shfl_sync` for broadcasting and reduction eliminates shared memory overhead and reduces atomic contention, providing both performance and code simplicity benefits.
- **Algorithm-hardware co-design:** Mapping the parallelization hierarchy to match GPU execution model (pixel-per-warp, RX-per-thread) maximizes occupancy and enables effective cooperation among threads.
- **CPU parallelization limitations:** While OpenMP provides a $6.2\times$ speedup on 12 cores, it encounters fundamental limitations from memory bandwidth contention and synchronization overhead. The GPU’s massive parallelism (thousands of threads) and specialized memory hierarchy overcome these limitations, achieving $10\times$ better performance than multi-core CPU parallelization.
- **Memory-bounded:** The low arithmetic intensity is fundamentally determined by the algorithmic nature of the method.

The comparison between OpenMP and CUDA implementations highlights the distinct advantages of GPU architecture for massively parallel, memory-intensive workloads. While multi-threading on CPUs provides incremental improvements, GPU acceleration enables order-of-magnitude performance gains through architectural features specifically designed for parallel computation.

5 Work Distribution

Name	Student ID	Key Responsibilities
Cheng-Po Chi	r12945010	Sequential/OpenMP implementation, performance benchmarking, and Parallelized code optimization,Ultrasound data.
Chin-Hui Chu	r12944041	OpenMP implementation, parallelized code optimization, and memory layout transformation.
Yi-Hua Chuang	b10201029	Parallelized code optimization, memory coalescing, and ablation study.

Table 6: Detailed task allocation and member contributions.